EXPERIMENT-3 (Skill)

Name: Simran Jangid

Rollno: 2520030166

Section: 8

PART - 1

Title: Implementation of Command History and Dynamic Memory Management

Aim: To implement command history using escape sequences and dynamic memory management using resizable buffers and linked lists.

Objective:

- To apply escape sequences for previous and next command navigation.
- To store command history and test recall functionality.
- To update the input buffer while navigating through history.
- To allocate buffers dynamically and resize arrays when required.
- To prevent buffer overflow and manage linked lists safely.
- To release memory correctly and verify memory usage with Valgrind.

Theory:

Command history allows a shell to remember previously entered commands so that the user can recall and reuse them. Terminal escape sequences are used to detect special keys such as the Up Arrow and Down Arrow. The Up Arrow is represented by ESC [A and the Down Arrow by ESC [B.

The shell maintains an input buffer for the command currently being edited. When a history entry is recalled, the input buffer is cleared and updated with the selected command. Dynamic memory allocation allows the history array and input buffer to grow when their capacity is reached.

Dynamic memory management uses malloc(), realloc(), and free(). A linked list can also store commands dynamically. Proper allocation, resizing, and deallocation help

prevent buffer overflow and memory leaks. Valgrind can verify that allocated memory is released correctly.

Algorithm:

Start the program.
Allocate an input buffer and a history array dynamically.
Read characters from the terminal.
Detect the Up Arrow and Down Arrow escape sequences.
Move backward or forward through stored commands.
Copy the selected history entry into the input buffer.
Resize the input buffer when its capacity is reached.
Store each completed command in dynamically allocated history.
Release the input buffer and history memory.
Run the program with Valgrind to check for memory leaks.
Stop.

Functions Used:

Function	Purpose
malloc()	Allocate memory dynamically
realloc()	Resize an allocated buffer or array
free()	Release dynamically allocated memory
strlen()	Find the length of a command
strcpy()	Copy a history command into the input buffer
read()	Read characters from the terminal

Sample Input: ls → Up Arrow → Enter

Sample Output:

shell> ls

file1.txt file2.txt shell.c

shell> [Up Arrow]

shell> ls

file1.txt file2.txt shell.c

Command recalled successfully.

Observation:

1. Escape sequences were detected for command navigation.

2. Previous commands were stored in history.

3. The Up Arrow successfully recalled the previous command.

4. The Down Arrow moved forward through the stored history.

5. The input buffer was updated with recalled commands.

6. Dynamic buffers were resized when additional space was required.

7. Memory was released correctly and verified using Valgrind.

Result:

The command history and dynamic memory management functionality was successfully implemented using C. Commands were stored and recalled using terminal escape sequences, the input buffer was updated dynamically, and allocated memory was released correctly.

Part 2:

Aim: To allocate buffers dynamically, resize arrays, prevent buffer overflow, manage linked lists, release memory correctly, and verify the program using Valgrind.

Objectives:

- To allocate memory dynamically using malloc().
- To resize arrays using realloc().
- To prevent buffer overflow by checking buffer capacity.
- To create and manage a linked list dynamically.
- To release all allocated memory using free().
- To verify memory leaks and invalid memory access using Valgrind.

Theory:

Dynamic memory allocation allows a program to request memory during execution instead of using a fixed-size array. When the allocated buffer becomes full, realloc()

increases its capacity. Checking size against capacity before writing prevents buffer overflow.

A linked list stores data in dynamically allocated nodes. Each node contains data and a pointer to the next node. Every node created using malloc() must eventually be released using free(). Valgrind helps identify memory leaks and invalid memory access.

Algorithm:

Start the program.
Allocate an initial buffer using malloc().
Insert values while checking buffer capacity.
Use realloc() when the buffer becomes full.
Create linked-list nodes dynamically.
Link each new node to the previous node.
Traverse and display the stored values.
Free every linked-list node.
Free the dynamic array and buffer.
Run the program using Valgrind and check the memory report.
Stop.

Functions Used:

Function	Purpose
malloc()	Allocate memory for buffers and nodes
realloc()	Increase the size of a dynamic array
free()	Release allocated memory
sizeof()	Determine the memory size required
printf()	Display stored values and results

Sample Input: 10 20 30 40 50

Sample Output:

Dynamic array: 10 20 30 40 50

Linked list: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Buffer resized successfully.

All allocated memory released successfully.

Observation:

1. Memory was allocated dynamically for the required data.

2. The array was resized when its capacity was reached.

3. Capacity checks prevented writing beyond the allocated buffer.

4. Linked-list nodes were created and connected successfully.

5. All dynamically allocated memory was released.

6. Valgrind was used to verify that no memory leaks remained.

Conclusion: The experiment successfully demonstrated command history, escape-sequence based navigation, dynamic buffer resizing, linked-list memory management, and correct memory deallocation. The use of Valgrind helped verify that the program handled dynamically allocated memory correctly.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>


#define INITIAL_SIZE 32


int main() {

  char **history = NULL;

  int history_count = 0, history_capacity = 4;

  char *buffer = malloc(INITIAL_SIZE);

  int capacity = INITIAL_SIZE, len = 0, index = -1;


  history = malloc(history_capacity * sizeof(char *));

  if (!buffer || !history) return 1;
```

```
printf("shell> ");
char c;

while (read(STDIN_FILENO, &c, 1) == 1) {
   if (c == 27) {
      char seq[2];
      if (read(STDIN_FILENO, &seq[0], 1) &&
         read(STDIN_FILENO, &seq[1], 1) &&
         seq[0] == '[') {
         if (seq[1] == 'A' && history_count > 0) {
            if (index == -1) index = history_count - 1;
            else if (index > 0) index--;
            strcpy(buffer, history[index]);
            len = strlen(buffer);
            printf("\rshell> %s", buffer);
         }
         else if (seq[1] == 'B' && history_count > 0) {
            if (index < history_count - 1) index++;
            else index = -1;
            printf("\rshell> ");
            if (index != -1) {
               strcpy(buffer, history[index]);
               len = strlen(buffer);
               printf("%s", buffer);
            } else len = 0;
         }
      }
```

```
    }
    else if (c == '\n') {
        buffer[len] = '\0';
        if (len > 0) {
            if (history_count == history_capacity) {
                history_capacity *= 2;
                history = realloc(history, history_capacity * sizeof(char *));
            }
            history[history_count] = malloc(len + 1);
            strcpy(history[history_count], buffer);
            history_count++;
        }
        printf("\nCommand stored.\nshell> ");
        len = 0; index = -1;
    }
    else if (c == 127) {
        if (len > 0) { len--; printf("\b \b"); }
    }
    else {
        if (len + 1 >= capacity) {
            capacity *= 2;
            buffer = realloc(buffer, capacity);
        }
        buffer[len++] = c;
        write(STDOUT_FILENO, &c, 1);
    }
}
```

```
    for (int i = 0; i < history_count; i++) free(history[i]);
    free(history);
    free(buffer);
    return 0;
}
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main() {
    int capacity = 2, size = 0;
    int *array = malloc(capacity * sizeof(int));
    Node *head = NULL, *tail = NULL;

    if (!array) return 1;

    for (int i = 1; i <= 5; i++) {
        if (size == capacity) {
            capacity *= 2;
            array = realloc(array, capacity * sizeof(int));
        }

        array[size++] = i * 10;
```

```
    Node *newNode = malloc(sizeof(Node));
    newNode->data = i * 10;
    newNode->next = NULL;

    if (head == NULL) head = tail = newNode;
    else { tail->next = newNode; tail = newNode; }
  }

  printf("Dynamic array: ");
  for (int i = 0; i < size; i++) printf("%d ", array[i]);

  printf("\nLinked list: ");
  for (Node *p = head; p != NULL; p = p->next)
    printf("%d -> ", p->data);
  printf("NULL\n");

  while (head != NULL) {
    Node *temp = head;
    head = head->next;
    free(temp);
  }

  free(array);
  return 0;
}
```

OUTPUT:

```
Last login: Sat Aug  8 15:28:02 on ttys000
[7rainreddy7@Gunapatis-MacBook-Air ~ % mkdir OS_Experiment
mkdir: OS_Experiment: File exists
[7rainreddy7@Gunapatis-MacBook-Air ~ % cd OS_Experiment
[7rainreddy7@Gunapatis-MacBook-Air OS_Experiment % nano history.c
[7rainreddy7@Gunapatis-MacBook-Air OS_Experiment % gcc history.c -o history
[7rainreddy7@Gunapatis-MacBook-Air OS_Experiment % ./history
shell> ls
Command stored.
shell> pwd
Command stored.
shell> date
Command stored.
shell> date
Previous command recalled successfully.
shell> pwd
Previous command recalled successfully.
shell> ls
Previous command recalled successfully.
shell> pwd
Next command recalled successfully.
Command history and input buffer updated successfully.
```

VERIFICATION